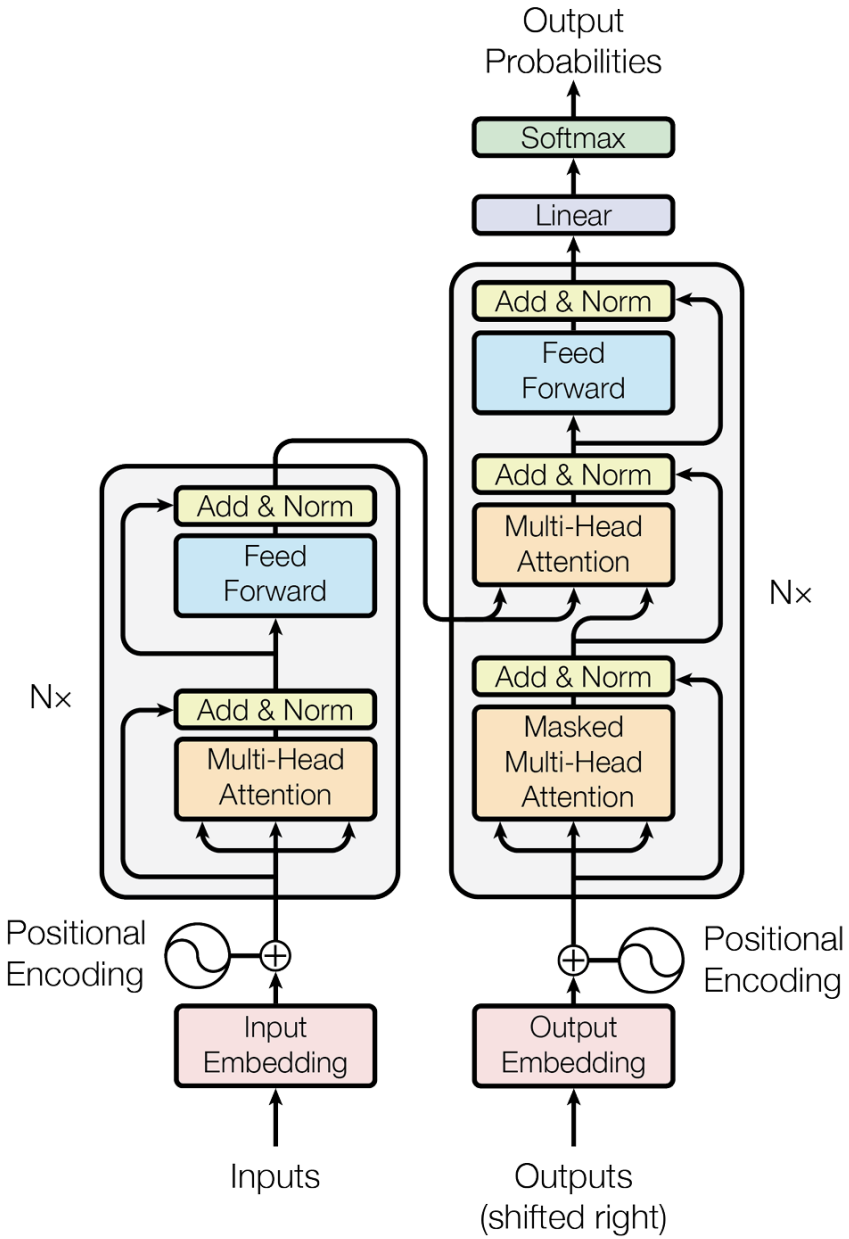
Transformers

Instructor: Chao Huang

Department of Computer Science
Montclair State University

Transformers



Overview

- Introduced in the paper "**Attention is All You Need**" by Google
- Primarily used for processing sequential data, especially in NLP tasks
- It uses self-attention mechanism, where the model weighs the importance of different words in a sentence
  - Multi-head attention: capture different relationships simultaneously
- Highly parallelizable, making them more efficient to train on large datasets compared to RNNs and LSTMs.
- Foundation for state-of-the-art models like BERT, GPT, and T5

How to Process Text Data?

- Consider this passage:
  - "The restaurant refused to serve me a ham sandwich because it only cooks vegetarian food. In the end, they just gave me two slides of bread. Their ambiance was just as good as the food and service."
- Q1: how do we feed them into a neural network?
  - One-hot encoding
  - Word embedding, e.g., of length 768 or 1024
- Q2: can MLP or CNN process them?
  - Input sizes vary
  - Sequential relationship

How to Process Text Data?

- Consider this passage:
  - "The restaurant refused to serve me a ham sandwich because it only cooks vegetarian food. In the end, they just gave me two slides of bread. Their ambiance was just as good as the food and service."
- Q1: how do we feed them into a neural network?
  - One-hot encoding
  - Word embedding, e.g., of length 768 or 1024
- Q2: can MLP or CNN process them?
  - Input sizes vary
  - Sequential relationship

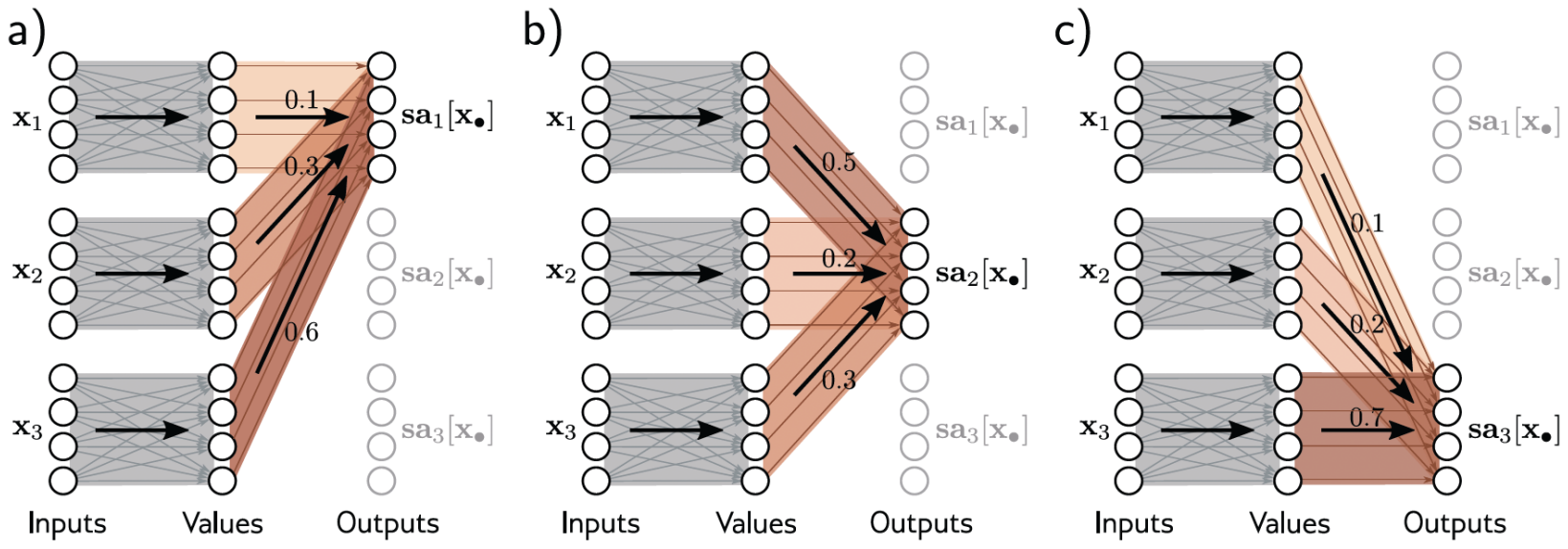
How to Process Text Data?

- Consider this passage:
  - "The restaurant refused to serve me a ham sandwich because it only cooks vegetarian food. In the end, they just gave me two slides of bread. Their ambiance was just as good as the food and service."
- Q1: how do we feed them into a neural network?
  - One-hot encoding
  - Word embedding, e.g., of length 768 or 1024
- Q2: can MLP or CNN process them?
  - Input sizes vary
  - Sequential relationship

How to Process Text Data?

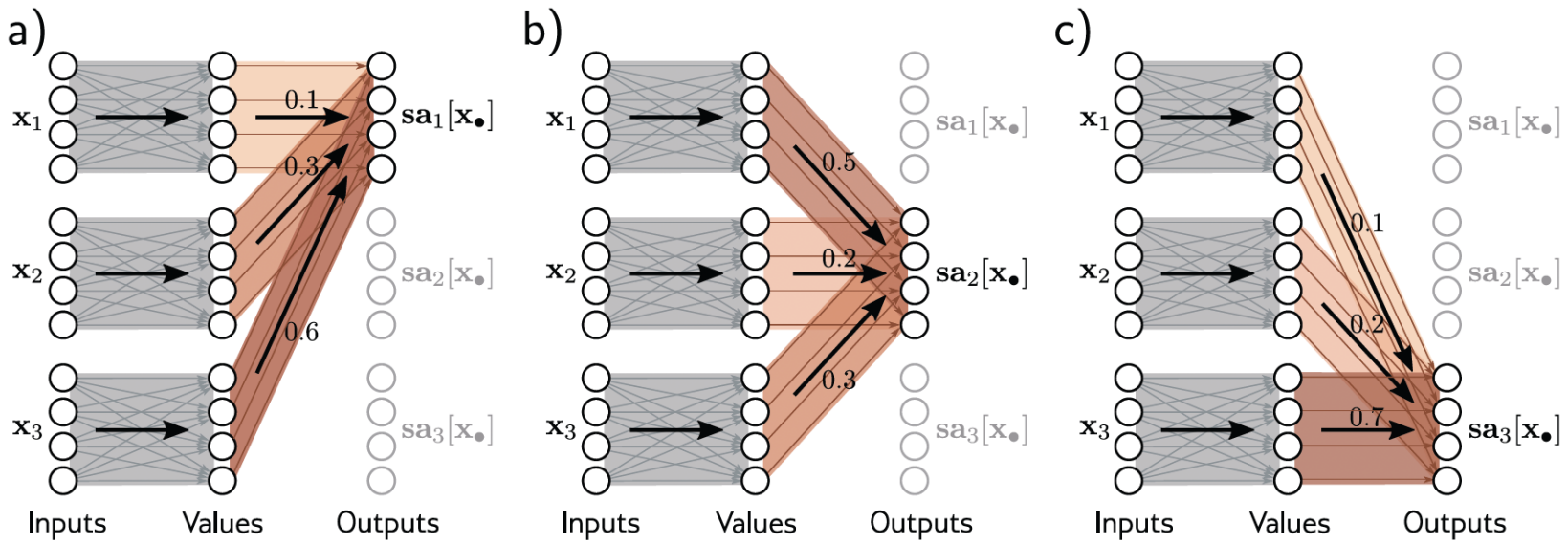
- Consider this passage:
  - "The restaurant refused to serve me a ham sandwich because it only cooks vegetarian food. In the end, they just gave me two slides of bread. Their ambiance was just as good as the food and service."
- Q1: how do we feed them into a neural network?
  - One-hot encoding
  - Word embedding, e.g., of length 768 or 1024
- Q2: can MLP or CNN process them?
  - Input sizes vary
  - Sequential relationship

Self-Attention



- Consider inputs $\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3$ with embedding $D = 4$
- $\mathbf{v}_m = \boldsymbol{\beta}_{\boldsymbol{v}} + \boldsymbol{\Omega}_v \boldsymbol{x}_m$ (possibly followed by ReLU)
- $\boldsymbol{sa}_n = \sum_{m=1}^{N} \underbrace{a[\boldsymbol{x}_m, \boldsymbol{x}_n]}_{\text{attention weights}} \boldsymbol{v}_m$
  - $a[\boldsymbol{x}_m, \boldsymbol{x}_n]$: attention that the nth output pays to input $\boldsymbol{x}_m$
  - $a[\cdot|\boldsymbol{x}_n]$: non-negative and sum to one
- Q: without ReLU, is this linear?

Self-Attention



- Consider inputs $\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_3$ with embedding $D = 4$
- $\mathbf{v}_m = \boldsymbol{\beta}_{\boldsymbol{v}} + \boldsymbol{\Omega}_v \boldsymbol{x}_m$ (possibly followed by ReLU)
- $\boldsymbol{sa}_n = \sum_{m=1}^{N} \underbrace{a[\boldsymbol{x}_m, \boldsymbol{x}_n]}_{\text{attention weights}} \boldsymbol{v}_m$
  - $a[\boldsymbol{x}_m, \boldsymbol{x}_n]$: attention that the nth output pays to input $\boldsymbol{x}_m$
  - $a[\cdot|\boldsymbol{x}_n]$: non-negative and sum to one
- Q: without ReLU, is this linear?

Where Do Attention Weights Come From?

- **Claim**: self-attention computation is non-linear
  - Attention weights are non-linear functions of inputs
- Let's apply two more linear transformations:
  - *Queries*: $\boldsymbol{q}_n = \boldsymbol{\beta}_q + \boldsymbol{\Omega}_q \boldsymbol{x}_n$
  - *Keys*: $\boldsymbol{k}_m = \boldsymbol{\beta}_k + \boldsymbol{\Omega}_k \boldsymbol{x}_m$
- Now we can compute attention weights:

$$a[\boldsymbol{x}_m, \boldsymbol{x}_n] = \text{softmax}_m[\boldsymbol{k}_\cdot^T \boldsymbol{q}_n] = \frac{\exp(\boldsymbol{k}_m^T \boldsymbol{q}_n)}{\sum_{m'=1}^{N} \exp(\boldsymbol{k}_{m'}^T \boldsymbol{q}_n)}$$

  - Softmax is non-linear
  - Dot product returns a measure of similarity between inputs
  - $a[\boldsymbol{x}_m, \boldsymbol{x}_n]$: attention that the nth output pays to input $\boldsymbol{x}_m$
    - ★ It depends on the relative similarities between the nth query to all keys

Why Dot Product Generates a Similarity Measure?

- $\boldsymbol{x} \cdot \boldsymbol{y} = |\boldsymbol{x}||\boldsymbol{y}| \cos(\theta)$, θ: angle between them
- **Example 1: Vectors with High Similarity**

$$\mathbf{a} = [1, 2], \quad \mathbf{b} = [2, 3]$$

So, $\mathbf{a} \cdot \mathbf{b} = (1 \times 2) + (2 \times 3) = 2 + 6 = 8$

- **Example 2: Orthogonal Vectors (No Similarity)**

$$\mathbf{c} = [1, 2], \quad \mathbf{d} = [-2, 1]$$

So $\mathbf{c} \cdot \mathbf{d} = (1 \times -2) + (2 \times 1) = -2 + 2 = 0$

- **Example 3: Very Dissimilar Vectors**

$$\mathbf{e} = [1, 2], \quad \mathbf{f} = [-10, -20]$$

So, $\mathbf{e} \cdot \mathbf{f} = (1 \times -10) + (2 \times -20) = -10 - 40 = -50$

Positional Encoding

- Q: does self-attention capture the order of inputs?
- No! Then what to do?
- Positional encoding
  - Add positional information to input before processing
  - Two types
    - Absolute positional encoding
    - Relative positional encoding, e.g., sinusoidal

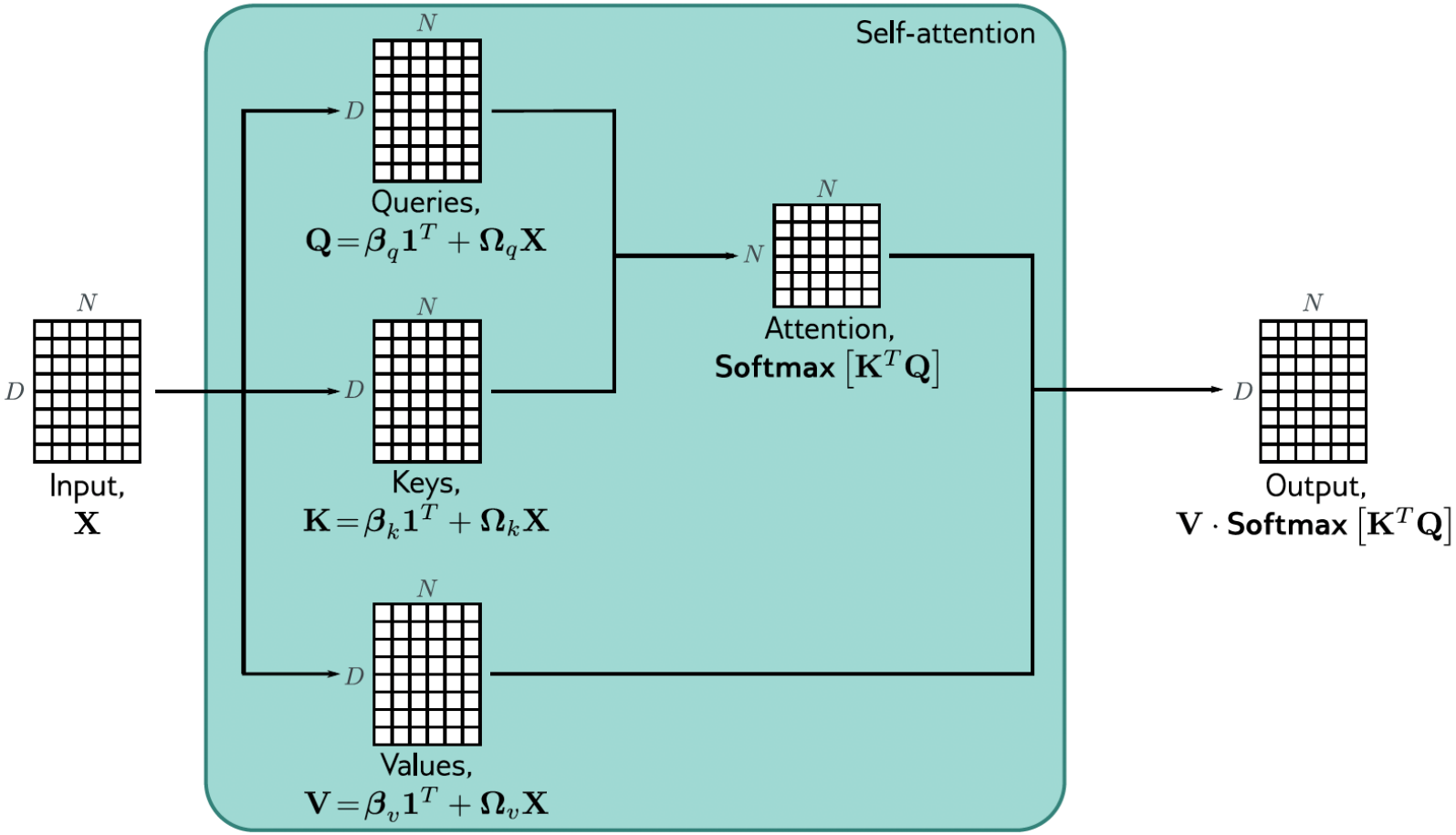
Positional Encoding

- Q: does self-attention capture the order of inputs?
- No! Then what to do?
- Positional encoding
  - Add positional information to input before processing
  - Two types
    - Absolute positional encoding
    - Relative positional encoding, e.g., sinusoidal

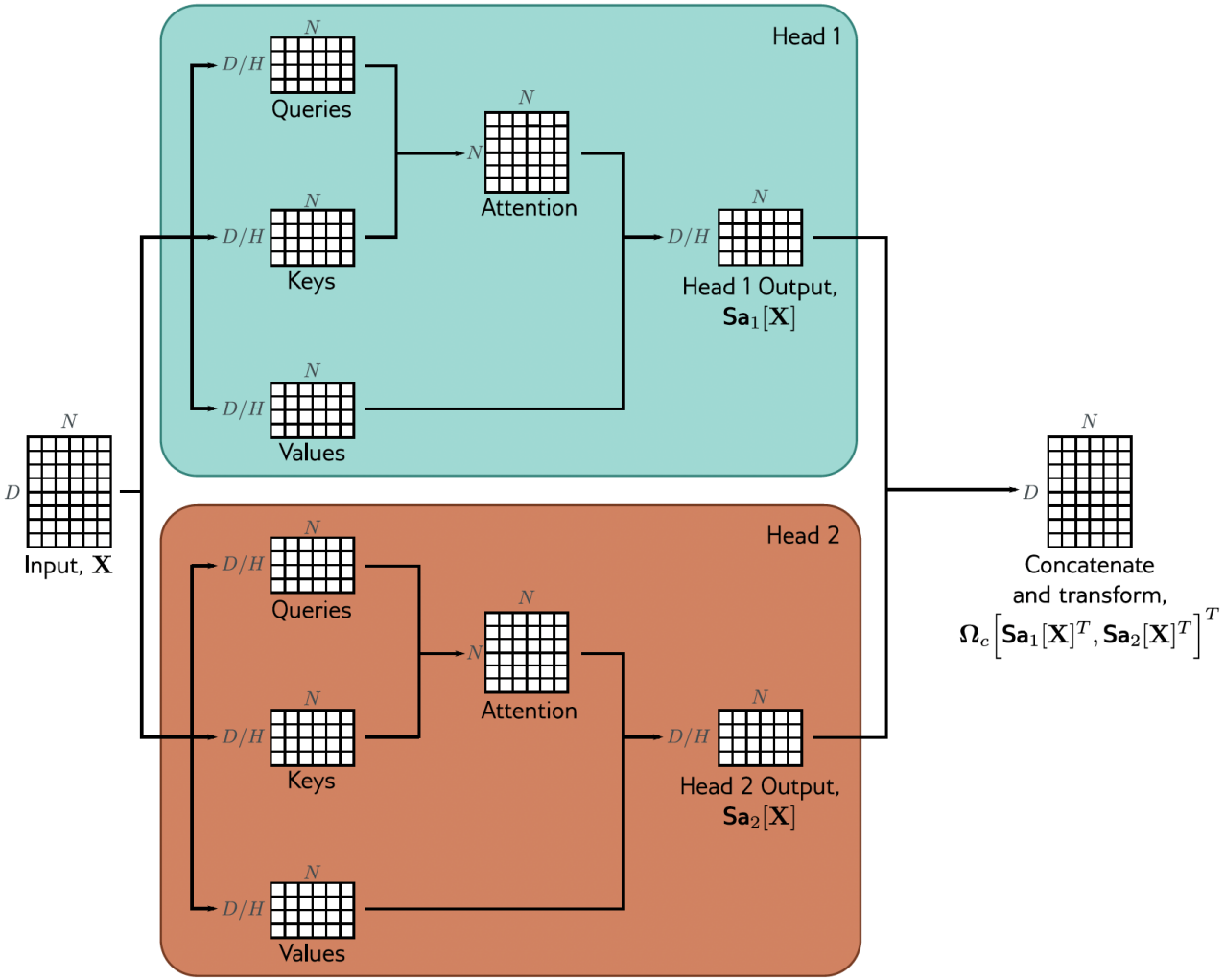
Positional Encoding

- Q: does self-attention capture the order of inputs?
- No! Then what to do?
- Positional encoding
  - Add positional information to input before processing
  - Two types
    - Absolute positional encoding
    - Relative positional encoding, e.g., sinusoidal

Self-Attention

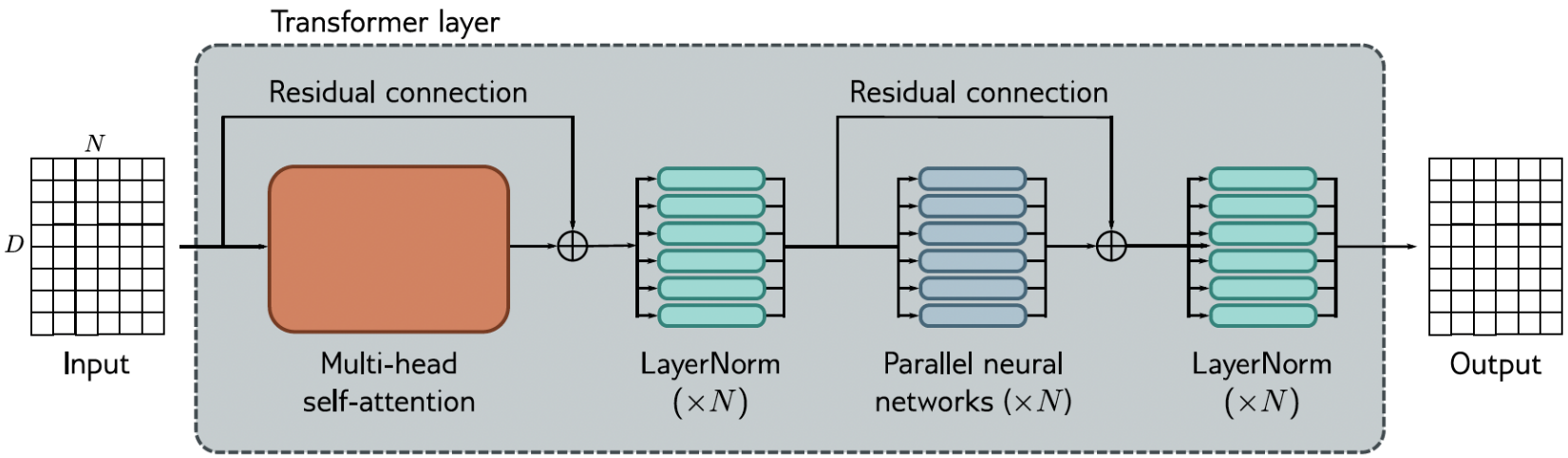


Multi-Head Attention



- Capture different types of sequential info, analogous to filters in CNN!
- Can be processed in parallel!

Transformer Layer



1. Multi-head attention: captures various sequential info
2. Residual connection
   - Allow the input to bypass the current layer and be added to its output
   - Promotes better gradient flow (e.g., mitigates vanishing) during BP
3. LayerNorm
   - Normalization by substracting mean and divided by standard deviation
   - Stable gradients and faster convergence
4. Parallel neural networks: just an MLP

Transformer Model

- Embedding matrix presenting the text is passed through a series of K transformer layers, called a transformer model
- Three types of transformer models
  - Encoder
    - Transform text embeddings into a representation that can support various tasks, e.g., BERT
  - Decoder
    - Predict next token to continue the input text, e.g., GPT3
  - Encoder-decoder
    - Used in senquence-to-sequence tasks, where one text string is converted into another, e.g., machine translation

What Are Encoder and Decoder?

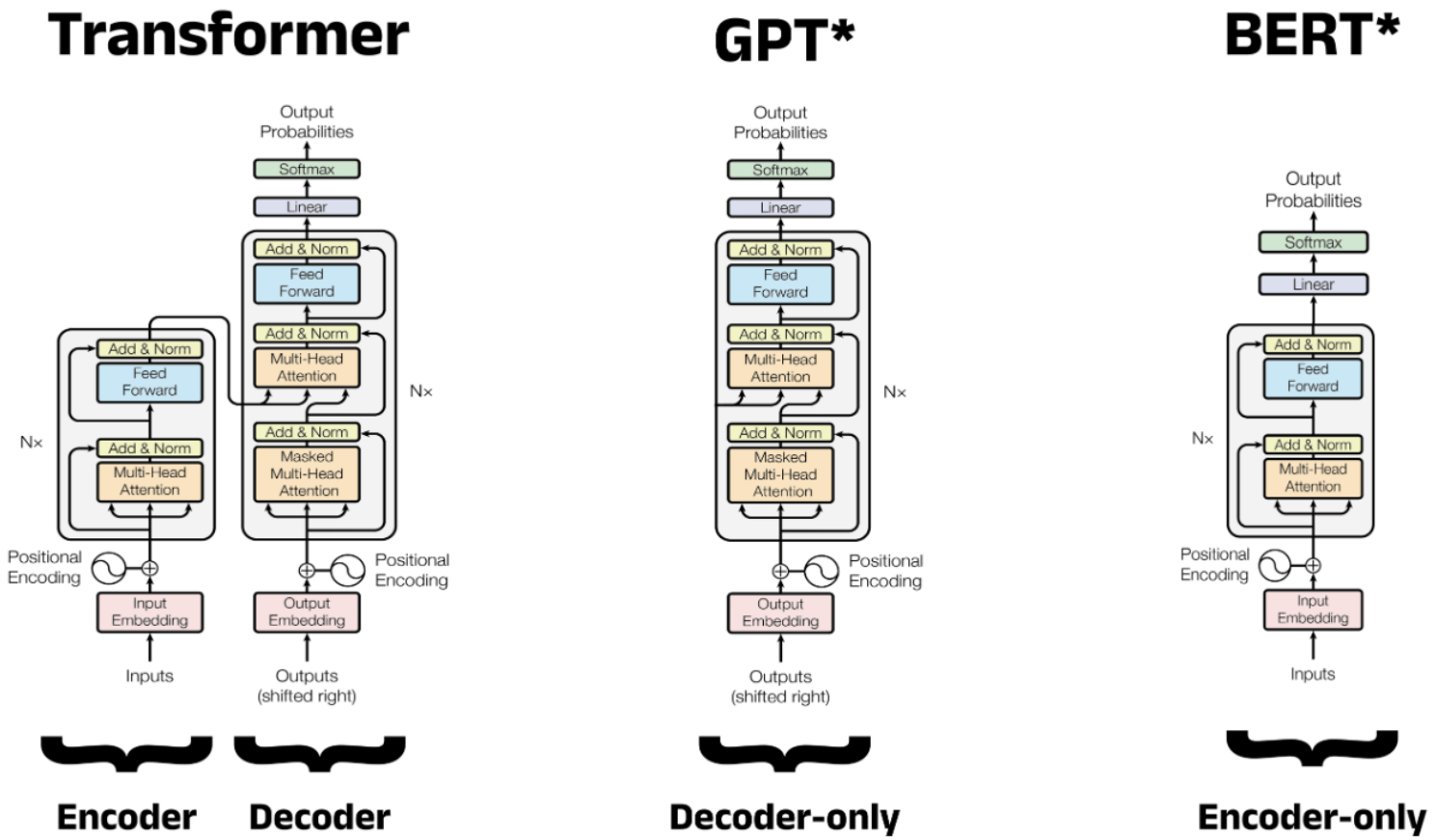


Figure: Illustrative example. Exact architectures might vary slightly.

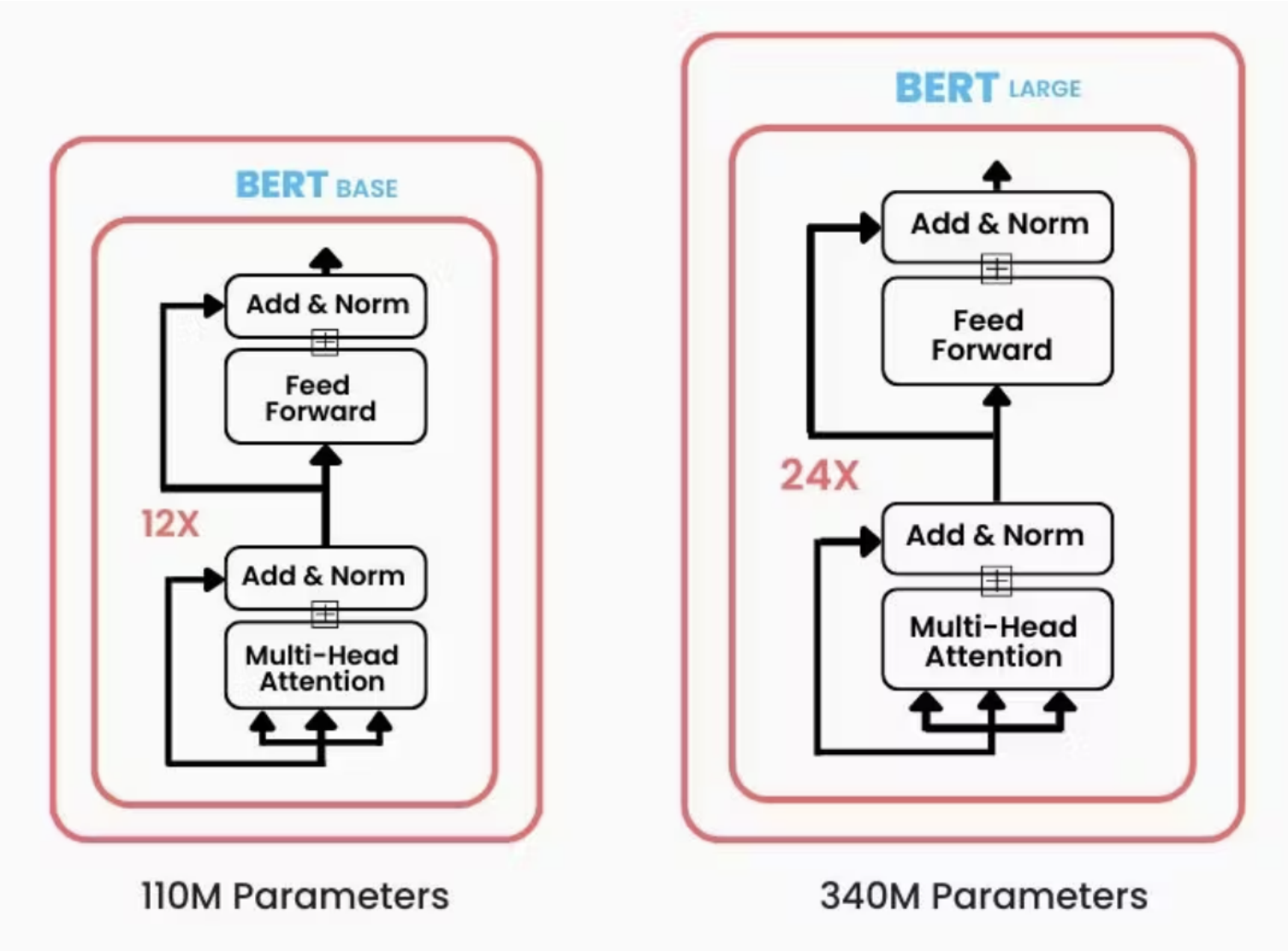
Encoder Example: BERT

- **BERT**: Bidirectional Encoder Representations from Transformers
  - Process text bidirectionally, instead of left-to-right or right-to-left
  - Structure
    - ★ Input converted to 1024-d word embedding
    - ★ 24 transformer layers, each contains self-attention with 16 heads
    - ★ The single hidden layer in fully connected networks is 4096-d
    - ★ #Parameters: 340 million (considered large when introduced in 2018)
- **Application Scenarios**
  - *Sentiment Analysis*
    - ★ Input: "I love this product!"
    - ★ Output: Positive sentiment.
  - *Named Entity Recognition (NER)*
    - ★ Input: "Apple is looking at buying U.K. startup for 1 billion."
    - ★ Output: Entities: [Apple (Organization), U.K. (Location), 1 billion (Money)]
  - .....

Encoder Example: BERT

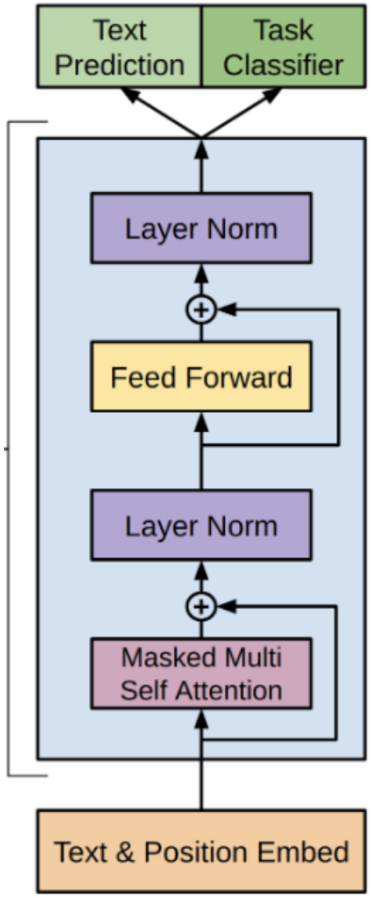
- **BERT**: Bidirectional Encoder Representations from Transformers
  - Process text bidirectionally, instead of left-to-right or right-to-left
  - Structure
    - ★ Input converted to 1024-d word embedding
    - ★ 24 transformer layers, each contains self-attention with 16 heads
    - ★ The single hidden layer in fully connected networks is 4096-d
    - ★ #Parameters: 340 million (considered large when introduced in 2018)
- **Application Scenarios**
  - *Sentiment Analysis*
    - ★ Input: "I love this product!"
    - ★ Output: Positive sentiment.
  - *Named Entity Recognition (NER)*
    - ★ Input: "Apple is looking at buying U.K. startup for 1 billion."
    - ★ Output: Entities: [Apple (Organization), U.K. (Location), 1 billion (Money)]
  - .....

BERT Structure



Decoder Example: GPT3

- Basic architecture extremely similar to encoder model
  - What is masked self-attention?
- Decoder: generate the next token in sequence!



Decoder: Language Modeling

- Consider: *It takes great courage to let yourself appear weak*

$$
\begin{aligned}
&Pr(\text{It takes great courage to let yourself appear weak}) \quad = \\
&\qquad Pr(\text{It}) \times Pr(\text{takes}|\text{It}) \times Pr(\text{great}|\text{It takes}) \times Pr(\text{courage}|\text{It takes great}) \times \\
&\qquad Pr(\text{to}|\text{It takes great courage}) \times Pr(\text{let}|\text{It takes great courage to}) \times \\
&\qquad Pr(\text{yourself}|\text{It takes great courage to let}) \times \\
&\qquad Pr(\text{appear}|\text{It takes great courage to let yourself}) \times \\
&\qquad Pr(\text{weak}|\text{It takes great courage to let yourself appear}). \qquad (12.14)
\end{aligned}
$$

- Predict next token $\rightarrow$ predict the conditional distributions $Pr(t_n|t_1, \cdots, t_{n-1})$ of each token given all prior tokens

Masked self-attention

- Training a decoder $\rightarrow$ finding para. that maximize sum of log conditional prob.
- When we pass the entire sentence, the term computing $Pr(great|it\ takes)$ would have access to the answer *great* and the right context *courage to let yourself appear weak*
  - The model can cheat rather than learn to predict the following words, and won't train properly
- Remedy: Mask certain tokens so that they can only attend current and previous tokens

GPT3 Statistics

Model size (paper name)	Parameters	Hidden size d_{model}	Layers n_{layer}
GPT-3 125 M	125 M	768	12
GPT-3 355 M	355 M	1 024	24
GPT-3 1.3 B	1.3 B	2 048	24
GPT-3 6.7 B	6.7 B	4 096	32
GPT-3 13 B	13 B	5 120	40
GPT-3 175 B (default "GPT-3")	**175 B**	**12 288**	**96**

Suggested Reading

- Simon J.D. Prince, "Understand Deep Learning", MIT press, `https://udlbook.github.io/udlbook/`
  - Chapter 12

Appendix: Encoder vs. Decoder Models

- **Encoder Models:**
  - **Function:** Processes input sequence and generates a context-aware representation (embedding).
  - **Example:** BERT (understands and represents text).
  - **Task:** Used for understanding tasks like classification, NER, and question answering.
- **Decoder Models:**
  - **Function:** Generates output sequence based on input representation (or previous outputs).
  - **Example:** GPT (generates text token by token).
  - **Task:** Used for generation tasks like translation, summarization, and text generation.
- **Encoder-Decoder Models:**
  - Combine both: Encoder for understanding input, Decoder for generating output.
  - **Example:** T5 (text-to-text transfer transformer), for tasks like machine translation.